

USER-MANUAL.md — how to drive it

For someone competent with a terminal who has never seen this codebase. Every command below was run on 2026-08-05 on Windows 11 (PowerShell + Git Bash, Python 3.12) unless it carries an `[UNVERIFIED – reason]` tag. The outputs shown are real outputs, not illustrations. Where a value will differ on your machine (a UUID, a path) that is called out.

o. Words you need first

There is no glossary document in this repository, so here are the terms this manual uses, defined once.

term	meaning
World	A complete specification of a counterfactual decade: inflation, policy rates, credit spreads, equity drift and volatility, crisis windows, private-market structure, plus a narrative the numbers never see. Stored as a JSON document validated against <code>schemas/worldspec-v1.2.schema.json</code> .
WorldSpec	The contract that document must satisfy. <code>schemas/</code> is read-only vendored truth — never edit it.
Preset	One of four ready-made worlds shipped in the repo: <code>stagflation</code> , <code>goldilocks</code> , <code>deflation_bust</code> , <code>reflation_boom</code> .
Engine	The simulator that turns a world into monthly returns for eight assets. The one that runs today is the deterministic toy engine, version <code>toy-v0.5</code> (was <code>toy-v0.3</code> when this manual's commands were executed; preset world ids moved from the 3xx to the 5xx block with it).
Path	One simulated history: 120 months of returns for every asset.
Ensemble	Many paths from one world, seeded <code>base_seed + 7919 · k</code> . The fan charts are quantiles across the ensemble.
RunRecord	The immutable receipt of one engine run — world id, seed, path count, and a SHA-256 digest of the outputs. Everything downstream is regenerated from it.
Replay	Recomputing a run's digest from its stored inputs and checking it matches. If it says <code>MATCH</code> , the numbers are bit-identical.
Reported vs true	Private sleeves (private equity, private credit, real estate) are marked by appraisal, so their <i>reported</i> returns are a smoothed version of the <i>true</i> ones. The gap is the point of the exercise, not a bug.
Sleeve	An asset bucket the institution holds: five liquid (equity, bonds, high yield, commodities, REITs) and three private (pe, pc, re).

Commitment / call / distribution	You promise a private fund money (commitment); it draws the money when it invests (capital call); it pays money back when it exits (distribution). Money you have promised but not yet paid is <i>unfunded</i> .
Coverage	Unfunded commitments divided by what you own. High coverage means you owe a lot relative to your assets. It looks healthier on reported marks than on true ones, exactly when it matters.
Forced sale	The institution ran out of cash and the waterfall sold something. Selling liquid holdings is ordinary funding; a forced secondary — selling private fund interests at a haircut — is distress.
The twin	The counterfactual you are scored against: the same institution, same world, same seed, that takes no action at any decision window.
Decision alpha	Your final value minus the twin's. Nine annual decision windows, four actions each.
Bundle	A single gzipped file (<1 MB) carrying everything the browser needs to render a world: the revealed path, the fan bands, the twin's ledger, and the wire.
Battery	A suite of statistical checks on generated data. There are two — see §7.

1. Setup

You need: Python 3.12 (the project pins it), [uv](#), and — only for the browser app — Node 20+. Verified here on Node v22.18.0, npm 10.9.3.

```
uv sync --dev          # creates .venv from uv.lock, exactly
```

[UNVERIFIED — the environment was already synced when I started and re-syncing mid-test-run would have disturbed it. Every `uv run ...` command below executed against the environment this produces.]

That is the whole bootstrap for the Python side. It installs pydantic, numpy, typer, duckdb, pyarrow, fastapi, uvicorn, and also **jax and torch**, which is why the first sync is large — the research generator layer needs them even though the playable path does not.

For the app:

```
cd app && npm install
```

[UNVERIFIED — `app/node_modules` was already populated; I ran the app's type check and test suite against it rather than reinstalling.]

There is no `.env` required for anything in this manual. `FRED_API_KEY` is only needed for `ah data refresh --live`, which this manual does not run.

Check the install:

```
$ uv run ah --version
0.1.0
```

```
$ uv run ah --help
Usage: ah [OPTIONS] COMMAND [ARGS]...

Alternate Histories platform CLI.

+- Options -----+
| --version          Show version and exit.                  |
| --db               <path> SQLite database path. [default: .../data/ah.db] |
| --help             Show this message and exit.            |
+-----+
+- Commands -----+
| run               Run the engine on a world and record a RunRecord (prints the |
|                   run_id).                                          |
| replay            Recompute a run's output digest and compare it to the stored |
|                   digest.                                          |
| inspect           Render a RunRecord's static figure page (wp5-02; regenerates + |
|                   verifies).                                       |
| bundle            Build the world bundle for a RunRecord (su-eng-01; DN-3 W2   |
|                   contract).                                       |
| credibility        Walk a set of worlds' numbers and flag what is not credible |
|                   (admin).                                         |
| verify            Verify a run reproduces its stored digest (prints True/False). |
| battery            Run the validation battery on a run's ensemble.      |
| chronicle          Print the append-only chronicle for a world.        |
| world              World lifecycle: build, validate, show.          |
| data               Data layer: refresh, status, asof, episode, intake.    |
| exp                Experiment tracking: list, show, diff.            |
+-----+
```

Work in a scratch database

Every `ah` command takes `--db`. The default is `data/ah.db` in the repo (which is gitignored). While you are learning, point at a scratch file so you cannot disturb anything:

```
export AH_DB=/tmp/ah-manual/ah.db      # any path you like; the dir is created for you
```

Every command below assumes `--db "$AH_DB"`. Drop it once you want the default.

2. Generate your first world

Two ways in: a preset, or a scenario sentence.

From a preset

```
$ uv run ah --db "$AH_DB" world build --preset stagflation
00000000-0000-4000-9000-000000000301
```

That single line of output is the **world id**. Preset ids are fixed, so you will get exactly this one. What just happened: the preset document was validated (schema, then coherence rules V1–V12), any out-of-bounds numbers were clamped, the world was stamped `validated`, saved to the store, and a `birth` entry was written to its append-only chronicle.

Check the validator's own account of it:

```
$ uv run ah --db "$AH_DB" world validate
world 00000000-0000-4000-9000-000000000301: clamps=0 warnings=[] blocking=[]
```

Zero clamps, zero warnings, nothing blocking — a clean world. (With no argument, every `ah` command that takes a world or run id uses the most recent one.)

Read the whole document if you want to see what a world is:

```
$ uv run ah --db "$AH_DB" world show 00000000-0000-4000-9000-000000000301
{
  "engine_defaults": {
    "base_seed": 771204,
    "generator_id": "toy-v0",
    "n_paths": 1000
  },
  "extensions": {},
  "factor_conditions": {
    "commodities": {
      "drift_annual_pct": 11.0
    },
    ...
  }
}
```

Run the engine

> **Name the world explicitly.** With no argument, `ah run` takes *the most > recently added world* — literally `ORDER BY rowid DESC LIMIT 1 > (cli.py:71-76)`. `ah replay`, `ah verify`, `ah chronicle`, `ah inspect`, > `ah bundle` and `ah world validate` all have the same fallback. It is > convenient and it is a trap: build a second world and every bare command > silently switches to it. Pass the id when it matters.

```
$ uv run ah --db "$AH_DB" run 00000000-0000-4000-9000-000000000301 --paths 200 --seed 771204
03a76c08-7219-41a9-b417-0b6566375582
```

Your run id will differ — it is a fresh UUID each time. Everything below uses `$RUN` for it:

```
export RUN=03a76c08-7219-41a9-b417-0b6566375582 # substitute yours
```

`--paths` and `--seed` are optional; without them the world's own `engine_defaults` are used (1000 paths, seed 771204 for stagflation). 200 paths keeps the later steps quick. The run simulated 200 histories of 120 months across eight assets, hashed the lot, and wrote a RunRecord plus a chronicle entry.

That is the decade generated. The rest of this manual is about looking at it and playing it.

> ****What you have *not* done.**** You have not used the hierarchical generator that > Step 2 promoted (`hier-flow-v1`). `ah run` runs the toy engine and nothing > else — see §8 and `docs/BUILD-SUMMARY.md` §5.1.

Aside: from a scenario sentence

You do not need this to follow the rest of the manual, and if you run it now every bare command below will start pointing at the world it creates rather than at stagflation. Come back to it after §6, or work in a separate `--db`.

Offline, the compiler is a fixture lookup over 50 recorded scenarios. The scenario text must match one of them; the catalogue is `fixtures/compiler/_manifest.json`.

```
$ uv run ah --db /tmp/ah-scenario/ah.db world build \
  --scenario "Valid scenario 00: inflation regime 0, rates path 0, vintage current."
00000000-0000-4000-8000-000000000000
```

The live compiler — free-text scenario through an Anthropic model — is the same command with `--live`. [UNVERIFIED — requires network and an API key; the test suite blocks sockets and this manual makes no live calls.]

3. Inspect what you made

Inspection today is **file- and CLI-based**. There is no admin GUI. The three surfaces are a static HTML figure page, a gzipped bundle you can read with any JSON tool, and the credibility console.

The chronicle — what happened to this world

```
$ uv run ah --db "$AH_DB" chronicle 00000000-0000-4000-9000-000000000301
[ 0] birth    {"clamps": 0, "status": "validated"}
[ 1] run      {"digest": "sha256:73326f13ed7b00d13ffb7fef527adcec716894e2065d9fb4495a77dd9351b4aa", "r
```

Append-only, enforced by a SQLite trigger. You cannot rewrite it.

The figure page — the path, the two planes, the episodes

```
$ uv run ah --db "$AH_DB" inspect $RUN --out /tmp/ah-manual/page.html
C:\...\page.html
```

125,911 bytes here, entirely self-contained (inline SVG, no external assets). Open it in a browser. It carries: a per-asset fan of cumulative-growth percentiles; the hold-course twin's value and sleeve weights; the **reported-vs-true toggle** for the three smoothed sleeves; episode annotations from the world's regime sequence and its narrative dispatches; and a pooled correlogram.

Two things worth knowing. The page is regenerated from the RunRecord alone and re-verifies the digest every time it renders — so a figure page is also a reproducibility check. And it is deterministic: render it twice, get identical bytes.

The bundle — everything the player's browser gets

```
$ uv run ah --db "$AH_DB" bundle $RUN --out /tmp/ah-manual/world.bundle.gz
C:\...\world.bundle.gz (31263 bytes compressed)      # a few bytes either way: the run_id UUID is insic
```

It is gzipped JSON, so you can read it directly:

```
$ python -c "
import gzip, json
d = json.loads(gzip.open('/tmp/ah-manual/world.bundle.gz').read())
print('version          : ', d['bundle_version'])
print('sections         : ', list(d))
print('digest_verified   : ', d['meta']['digest_verified'])
print('decision months   : ', d['summary']['decision_months'])
print('feed items        : ', len(d['feed']['artifacts']))
print('feed types        : ', sorted({i['type'] for i in d['feed']['artifacts']}))
print('twin ledger       : ', list(d['twin_ledger']))
"
version          : world-bundle-0.4
sections         : ['bands', 'bundle_version', 'feed', 'meta', 'revealed', 'summary', 'twin_ledger']
digest_verified  : True
decision months  : [11, 23, 35, 47, 59, 71, 83, 95, 107]
feed items       : 222
```

```
feed types      : ['cb_statement', 'newspaper', 'quarterly_statement', 'release_page', 'wire_digest']
twin ledger     : ['calls', 'cash', 'distributions', 'nav_reported', 'nav_true', 'private_weight_true',
```

Where to find each thing:

you want	look at
the revealed path (monthly returns per asset, plus reported variants)	<code>revealed.tape</code> , with <code>revealed.series_order</code> naming the columns, and <code>revealed.tape_seal</code> the SHA-256 over exactly those rounded bytes
the fan bands	<code>bands.<asset>.p5 / p25 / p50 / p75 / p95</code> — cumulative growth
the events / the wire	<code>feed.artifacts</code> , each item carrying the <code>month</code> it is revealed on; also <code>feed.dispatches</code> and <code>feed.chronicle</code>
the cashflows	<code>twin_ledger</code> — per closed quarter: <code>calls</code> , <code>distributions</code> , <code>cash</code> , <code>unfunded</code> , <code>nav_true</code> , <code>nav_reported</code> , <code>private_weight_true</code>
the regime episodes	<code>summary.episodes</code>



Read `twin_ledger`, not `summary.twin_final_value`, for the institution that scores. The two disagree, on purpose in the code's history and by accident in its present state —

`summary.twin_final_value` is computed on Step 0's toy institution (no cash account, no calls), `twin_ledger` on the real one. In this bundle they read 166.502 and 76.712 respectively. `docs/BUILD-SUMMARY.md` §5.2.

Inspect the generator's input data (added 2026-08-07, postdates this manual's survey)

```
uv run uvicorn ah.dataconsole:app --port 8796
```

Read-only over the vintage store. `/` answers "is anything missing or stale": every registered series with coverage, gaps, staleness against its SLA, and the share of history that is proxy-spliced. `/class/<name>` (equities, rates-bonds, credit, inflation-macro, fx, privates) shows the raw series feeding each asset class — proxy stretches shaded — and then the derived factors the generator actually consumes; the privates page overlays reported vs de-smoothed returns with both moment sets. `/series/<id>` is the drill-down (vintages, gaps, QC findings, manifest entry verbatim); `/factors` recomputes the whole factor panel mechanically and shades the sealed train/validation windows (holdout labeled SPENT). It writes nothing — a guard test enforces zero write call sites.

Compile a scenario in the browser (added 2026-08-06, postdates this manual's survey)

```
uv run uvicorn ah.buildconsole:app --port 8798
```

Type a scenario, press **Compile (dry-run)**, and watch the five-stage ledger (prompt → model → extract → validate → stamp) fill in live. Nothing is stored until you press **Keep** — which stamps and stores the world exactly as `ah world build` would, optionally records an engine run, and links you to the QA shelf on 8799. Every attempt, including failures with their raw payloads, lands in `data/buildconsole/attempts.jsonl`. Offline it replays `fixtures/compiler/` by scenario slug; the **live** checkbox needs `ANTHROPIC_API_KEY` — from the environment, or (as of 2026-08-07) falling back to the repo-root `.env`, the same pattern the data scripts use for `FRED_API_KEY`, so plain `uv run uvicorn ah.buildconsole:app --port 8798` works from a fresh shell. Live compiles work as of WP-A (prompt v2, envelope stamped by the system): a throwaway scenario ran all six stages green on 2026-08-06. The ledger shows exactly where and why anything fails.

The credibility console — is this world's arithmetic sane?

Admin tooling. It regenerates ensembles from stored seeds, computes decade statistics, and marks anything outside a **declared** plausible band. It writes nothing and cannot fail a build.

```
$ uv run ah --db "$AH_DB" credibility --preset stagflation --preset goldilocks \
  --paths 200 --out /tmp/ah-manual/credibility.html
C:\...\credibility.html (2 worlds, 2 flags)
```

Open it. Per asset you get annualized return and volatility over the decade against the declared band; the factor paths; the smoothing statistics; and — the part worth your attention — the **private-programme section**: the model's own call-rate and distribution curves, the commitment ladder year by year, the vintage NAV stack, and both tier-1 linkage multipliers with a same-tape linkage-off counterfactual beside them.

A flag is an invitation to look, not a failure. The bands are one allocator's priors, written down in `src/ah/credibility.py` and `src/ah/programme.py` so that disagreement is about a number rather than a vibe.

4. Play / step through a run

Play is driven by a **session service** — a small FastAPI app that owns everything that scores — plus the React app that renders it. The server is the authority; the browser never computes a result it could be trusted to fake.

Start the service

```
uv run uvicorn ah.serve:app --port 8787
```


Port 8787, not 8000. `app/vite.config.ts` proxies `/sessions` there. [The command itself is UNVERIFIED – an instance was already listening on 8787 when I began, and I queried it rather than starting a competing one. The app it serves is the one I exercised in full, on port 8788.]

This binds to the repo's default database (`data/ah.db`). If you built your world in a scratch database, point the service at it:

```
uv run python -c "  
import uvicorn  
from ah.serve import create_app  
uvicorn.run(create_app('/tmp/ah-manual/ah.db'), host='127.0.0.1', port=8788)  
"
```

That is the form I used to verify everything in this section, on port 8788, so that a working database was never touched. Both forms run the identical app.

Start the app

```
cd app && npm run dev      # vite on 5173
```

[The command itself is UNVERIFIED – a vite dev server was already on 5173; I queried it directly (/ served the SPA, `/sessions/*` proxied to the service) rather than starting a second one.]

Then open `http://localhost:5173` . The app loads a bundle, verifies its seal in the browser, shows the fan chart with the revealed path clipped to the reveal pointer, plays the wire in timeline, and presents each decision window.

Or drive it directly with HTTP

Every step the app takes is one request. This is the whole loop, verified end to end.

Open a session:

```
$ curl -s -X POST http://127.0.0.1:8788/sessions \  
  -H "content-type: application/json" \  
  -d '{"run_id":"'$RUN'", "basis": "reported"}'  
{  
  "session_id": "53741906-94dd-48a1-a418-a61e7f1706fb", "run_id": "03a76c08-...",  
  "world_id": "00000000-0000-4000-9000-000000000301", "months": 120,  
  "revealed_months": 0, "basis": "reported", "ranked": false, "participant": null,  
  "decisions": {}, "window_log": [], "status": "active", ...}
```

`basis` is `reported` (you see the smoothed private marks — the institutional status quo) or `actual` .
`ranked: true` plus a `participant` name writes to the leaderboard on completion; leave it off to

practise.

Advance the reveal pointer — it is monotone and capped; the server refuses to go backwards:

```
$ export SID=53741906-94dd-48a1-a418-a61e7f1706fb
$ curl -s -X POST http://127.0.0.1:8788/sessions/$SID/advance \
  -H "content-type: application/json" -d '{"to_month":12}'
```

The response is the session, marked to market **as at the pointer**. After the first year of the stagflation preset:

revealed_months	12	
value	98.0151	(the book, on the session's basis)
twin_value	98.0151	(identical – no decision taken yet)
cash	0.3183	
coverage_true	0.1782	(unfunded / true NAV)
private_weight_true	0.3436	(policy band is 0.15–0.40)
calls_paid	0.2161	
distributions_received	0.7864	
spending_paid	1.1646	
forced_sale_total	0.0	

Nothing is marked before month 3 — no quarter has closed.

Commit a decision. Windows are months 11, 23, 35, 47, 59, 71, 83, 95, 107; `GET /sessions/{sid}` returns them as `decision_windows`. Actions are `hold`, `derisk`, `leanin`, `secondary`. A decision is **final** — the server refuses to change one.

```
$ curl -s -X POST http://127.0.0.1:8788/sessions/$SID/decisions \
  -H "content-type: application/json" \
  -d '{"month":11,"action":"derisk"}'
# -> decisions: {"11": "derisk"}, status: active
```

What each action does to the real institution: `derisk` moves 10 points from the growth pair (equity, pe) to the defensive pair (bonds, pc) — **on the liquid leg only**, because a private fund's NAV is not a dial; `leanin` is the reverse; `secondary` sells 8 points of your largest live PE cohort for cash at a 19% haircut. A decision taken at window month `m` lands at the start of the following quarter.

Play out the decade — advance past each window, decide, repeat:

```
for m in 23 35 47 59 71 83 95 107; do
  curl -s -X POST http://127.0.0.1:8788/sessions/$SID/advance \
    -H "content-type: application/json" -d '{"to_month\":"$(m+1)}"' > /dev/null
  curl -s -X POST http://127.0.0.1:8788/sessions/$SID/decisions \
    -H "content-type: application/json" -d '{"month\":"$m","action\":"hold\}"' > /dev/null
done
```

```
curl -s -X POST http://127.0.0.1:8788/sessions/$SID/advance \
-H "content-type: application/json" -d '{"to_month":120}' > /dev/null
```

Close it out:

```
$ curl -s -X POST http://127.0.0.1:8788/sessions/$SID/complete
# -> status: completed, revealed_months: 120, 9 decisions
```

5. Score it

The outcome endpoint is available **only** on a completed session (it returns 409 otherwise). It is fully wired.

```
$ curl -s http://127.0.0.1:8788/sessions/$SID/outcome
```

From the session played above — de-risk at the first window, hold thereafter:

```
alpha                : 3.3735227993676062
final_value          : 80.08573667500643
twin_final_value     : 76.71221387563882
decision_alpha_version: port-v1-cashflow
forced_secondaries   : 0
windows              : [(11, 'derisk', 3.3735), (23, 'hold', 0.0), (35, 'hold', 0.0),
                        (47, 'hold', 0.0), (59, 'hold', 0.0), (71, 'hold', 0.0),
                        (83, 'hold', 0.0), (95, 'hold', 0.0), (107, 'hold', 0.0)]
sum of contributions : 3.3735227993676062
series               : active[], twin[], drift_twin = null
```

Read that carefully, because it is the whole scoring model:

- **alpha is your final value minus the twin's.** The twin is the same

institution on the same tape that acted at no window. Here: +3.37 points on a book that started at 100.

- **Per-window contributions are exact, not sampled.** Contribution j is the

value of playing your decisions up to and including window j (holding after) minus the value of the prefix before it. The chain telescopes, so the parts sum to the whole *by construction* — and indeed 3.3735227993676062 appears twice above, once as alpha and once as the sum. It costs $K+1$ full simulations for K windows and the server does all of them.

- **The eight hold windows contributed exactly 0.0.** Correct: holding is what

the twin does, so it adds nothing over the twin by definition.

- `decision_alpha_version` is `port-v1-cashflow`. This is the *product's*

alpha, computed on the Step-3 institution with the cash account. It is deliberately **not** `ah.eval.decision_metrics.DECISION_ALPHA_VERSION ( "1.0" )`, which names Step 5's research definition, is measured in log points, and sits inside a cryptographic seal. Scores under the two never share a table.

- `drift_twin` is `null`. Three series are in the contract; only two have

data. That is by design — the slot exists so the third's arrival is a data change, not a redesign.

The leaderboard

Ranked sessions (`"ranked": true` with a `participant` name) write one row on completion, keyed on (world_id, seed, decision_alpha_version, participant). First play stands; a replay cannot overwrite it.

```
$ curl -s "http://127.0.0.1:8788/leaderboard/<WORLD_ID>?seed=771204&alpha_version=port-v1-cashflow"
{"world_id":"...", "seed":771204, "decision_alpha_version":"port-v1-cashflow", "rows":[]}
```

The triple key is **required** in the query — boards never mix worlds, seeds, or scoring versions.

⚠ **This endpoint does not work through the dev app.** `vite.config.ts` proxies only `/sessions`, so the browser's `/leaderboard/...` request hits vite and gets the SPA's HTML back; the component renders an error where the board should be. The server is fine — call it directly, as above. Fix and evidence in `docs/BUILD-SUMMARY.md` §5.4.

6. Replay and verify

This is the property the whole system is built around: any run can be reproduced, bit for bit, from its record.

```
$ uv run ah --db "$AH_DB" replay $RUN
stored : sha256:73326f13ed7b00d13ffb7fef527adcec716894e2065d9fb4495a77dd9351b4aa
replay : sha256:73326f13ed7b00d13ffb7fef527adcec716894e2065d9fb4495a77dd9351b4aa
MATCH
```

`MATCH` means the engine was re-run from the stored world, seed and path count, and the recomputed SHA-256 over the output tensors is identical. Anything else exits non-zero.

The short form, for scripts:

```
$ uv run ah --db "$AH_DB" verify $RUN
True
```

Two things make this hold: every random number comes from a single

`numpy.random.Generator(PCG64(seed))` with all draws taken up front in a fixed order, and nothing anywhere in the numeric path reads a clock. If you change the engine's numbers, `TOY_ENGINE_VERSION` must be bumped and worlds moved to a new `world_id` block, so scores from two engines cannot share a leaderboard row.

`ah inspect` and `ah bundle` both run this same verification on every render — a tampered record produces a loud `digest_verified: false` rather than a pretty page.

7. Run the tests

The everyday suite

```
uv run pytest
```

121 modules, **2,226 tests**. It is not fast — about 20 minutes here, because the generator and evaluation suites pull in torch and jax. Current result on `main`: **all 2,226 passed, exit 0**.

Note that `-q` is already in `addopts`, so adding your own `-q` makes it `-qq` and suppresses the summary line entirely. Run it plain, or read the exit code.

For a quick confidence check after an install, run the core rails only — this is the everyday command:

```
$ uv run pytest tests/test_engine.py tests/test_validator.py tests/test_worldspec.py \
               tests/test_stores.py tests/test_digest.py tests/test_g0_end_to_end.py
80 passed in 3.29s
```

A single test, by node id or by keyword:

```
$ uv run pytest tests/test_engine.py::test_golden_snapshot
1 passed in 0.53s

$ uv run pytest -k desmooth -q
..... [100%]
```

The browser app has its own suite:

```
cd app && npm run typecheck && npm run test
# -> Test Files 8 passed (8) / Tests 32 passed (32)
```

And the static checks:

```
uv run ruff check .          # -> All checks passed!
uv run pyright               # -> 0 errors, 0 warnings, 0 informations
uv run ruff format --check . # -> currently FAILS on two markdown plan docs
```

That last one is a real, current failure on `main`, not something you caused: `ruff 0.16` formats Python code blocks inside Markdown and two committed plan documents have unformatted blocks. `uv run ruff format .` fixes it. `docs/BUILD-SUMMARY.md` §5.5.

No test in this repository may touch the network — `pytest-socket` is enabled via `addopts` and will fail any test that tries.

The validation battery — do not run it

There are two things called "the battery" and they are different objects.

- **The Step-0 stylized battery** — `uv run ah battery [RUN_ID]` or

`uv run python -m ah.battery.report`. It computes excess kurtosis, skewness, a Hill tail index, return and absolute-return autocorrelations, a drawdown distribution and a correlation distance over an ensemble, and compares each to `src/ah/battery/thresholds.yaml`.

Battery execution is gated pending threshold ratification, and this manual does not run it. The thresholds file says so itself: every metric is `status: todo`, described in the file as "placeholders documenting intent, not ratified thresholds". Because a report only fails on `enforce`-level breaches and no metric is `enforce`, the battery cannot currently fail — so a green run would tell you nothing, and a number read off it would be a number nobody has agreed to. Until the thresholds are ratified, treat it as plumbing.

- **The Step-2 validation battery** — `ah/eval/battery.py`, with metric suites

under `ah/eval/metrics/`. This one has real thresholds, sealed together with the code that judges them in `pre-registration.lock`. It is research machinery, run by the scripts under `scripts/`, and its committed results live in `artifacts/`. Its thresholds cannot be edited without going through the machine-checked amendment log.

If you are checking that your install works, §7's first block is the right test. The battery is not that.

8. Troubleshooting

1. "The session service is down" — but you checked port 8000. It listens on **8787**.

`app/vite.config.ts` proxies `/sessions` there, and nothing in the project uses 8000. Check with `Get-NetTCPConnection -LocalPort 8787 -State Listen` (PowerShell). If you need a second instance on another port, use the `create_app(...)` form in §4.

2. You edited `ah/serve.py` and nothing changed. Uvicorn without `--reload` keeps serving the module it imported at start-up, so your live checks silently exercise stale code. Kill it and restart. `pkill` does not exist on Windows:

```
Get-NetTCPConnection -LocalPort 8787 -State Listen |  
ForEach-Object { Stop-Process -Id $_.OwningProcess -Force }
```

[The `Get-NetTCPConnection` half is verified – it correctly reported the listeners on 8787 and 5173. The `Stop-Process` half is UNVERIFIED: killing a running service is not something to do while documenting it.]

3. Invalid value: no worlds in the database yet (or no run records...). Every command that takes an optional id falls back to "the most recent one", and the fallback fails on an empty database. Either you pointed `--db` somewhere new, or you skipped a step. Build a world first:

```
$ uv run ah --db /tmp/fresh.db run  
Usage: ah run [OPTIONS] [world_id]  
Try 'ah run --help' for help.  
+- Error -----+  
| Invalid value: no worlds in the database yet           |  
+-----+  
+-----+
```

The same shape of error names your options when you mistype a preset: `unknown preset 'nope'`. Available: `deflation_bust`, `goldilocks`, `reflation_boom`, `stagflation`.

4. `UnsupportedGeneratorError: engine implements only 'toy-v0'`. You set `engine_defaults.generator_id` to something the CLI cannot run:

```
UnsupportedGeneratorError: engine implements only 'toy-v0', got 'hier-flow-v1'.  
Other generators arrive in later steps.
```

This is not a misconfiguration you can fix in the world document. The hierarchical generators live in `ah/gen/` and are reachable only through the research scripts; `ah run`, `ah bundle` and the session service all run the toy engine. Additionally, the generator stack requires SHA-pinned fitted artifacts and trained checkpoints under `experiments/`, which is gitignored — on a fresh clone it cannot be instantiated at all, only refitted and retrained. `docs/BUILD-SUMMARY.md §5.1`.

5. The leaderboard shows an error string in the app. Expected today. Vite proxies `/sessions` and not `/leaderboard`, so the fetch gets HTML and the JSON parse throws; the component catches it and renders the message. Query the service directly (§5) until the proxy gains a second entry.

6. `ruff format --check` . fails and you did not touch any Python. Two Markdown plan documents under `docs/superpowers/plans/` contain Python code blocks that ruff 0.16 wants reformatted. `uv run ruff format` . fixes them.

7. **BundleError: bundle is N bytes compressed, over W2's 1000000-byte budget – shrink it, do not ship it.** [UNVERIFIED – read from `bundle.py:190` ; I did not build a bundle large enough to trigger it.] The bundle refuses to be built rather than shipping something that will poison load times. Measured on the 200-path stagflation bundle above (130,569 bytes uncompressed), the weight sits in `feed` (55.3%) and `bands` (33.0%), so a shorter horizon or a quieter world is the lever, not the path count — the bands are quantiles, whose size depends on months and assets, not on `n_paths` . That bundle compresses to ~31 KB, so you have to work at it to hit the 1 MB budget.

8. **A non-ASCII character in CLI output.** `ah data episode 2022` emits an em-dash (`data/episode.py:107` , echoed at `data/cli.py:154`). It does not crash — exit 0 either way — but **it renders correctly in PowerShell and as ? in Git Bash**, which is exactly the class of breakage the project's ASCII-in-CLI-output convention exists to prevent. If you see a mangled character or a `UnicodeEncodeError` , set `PYTHONIOENCODING=utf-8` or redirect the output to a file.

Closing note

Corrections made after review

This manual was fact-checked by re-running its commands in a clean scratch database. Two things were wrong in the first draft and are fixed above, recorded here because a manual that hides its own errata is not trustworthy:

1. **§2 built the scenario world before running the engine.** Since `ah run` with no world id takes the most recently added world (`ORDER BY rowid DESC`), a reader following the steps literally would have run the *scenario* world and got digest `7798964d...` instead of the `73326f13...` shown. The scenario build is now an aside after §6, the run names its world explicitly, and the fallback is documented as the trap it is. Verified both ways: implicit → `7798964d...` , explicit world id → `73326f13...` . 2. **§8's em-dash note was too reassuring** — the character does mangle in Git Bash, which is the failure the convention exists to prevent.

Commands that failed when I tried them

- `uv run ruff format --check .` — **fails**, 2 files would be reformatted

(`docs/superpowers/plans/2026-08-04-private-programme-console-section.md` , `docs/superpowers/plans/2026-08-05-world-and-wire-audit.md`). This is a real failure on `main` , reproduced with the locked ruff 0.16.0.

- `GET /leaderboard/...` **through the vite dev server** — returns `text/html`

(the SPA fallback) instead of proxying to 8787. The same request direct to the service returns correct JSON.

- Nothing else I ran failed. For the record of what *did* pass: the full Python

suite (2,226 tests, exit 0), the core-rails subset (80 in 3.29s), the app suite (32 tests), `ruff check`, and `pyright` (0 errors).

Commands documented but not executed

Marked inline where they appear, gathered here: `uv sync --dev` and `npm install` (both environments were already provisioned and re-provisioning mid-run would have disturbed the test suite); `uvicorn ah.serve:app --port 8787` and `npm run dev` (instances were already listening on 8787 and 5173, so I queried those rather than starting competitors — the identical FastAPI app was exercised in full on port 8788 against a scratch database); `ah world build --live`; the `Stop-Process` half of the restart recipe; and the bundle size-budget error.

Capabilities I could not verify

capability	why not
The Step-o validation battery (<code>ah battery</code> , <code>python -m ah.battery.report</code>)	Excluded by instruction: battery execution is gated pending threshold ratification.
The live compiler (<code>ah world build --live</code>)	Needs network and an API key.
<code>ah data refresh --live</code>	Needs network and <code>FRED_API_KEY</code> . Offline refresh from fixtures exists but I did not exercise it.
Tier-2 authoring end to end (<code>scripts/run_authoring_regression.py</code>)	Live model calls. I read the committed evidence (30/30, 100.0%) rather than reproducing it.
Any neural generator actually <i>generating</i>	<code>registry.resolve("hier-flow-v1")</code> resolves here and the checkpoint SHA matches the pin — I verified that much. Running a full generation is GPU-scale work and would not reproduce on a clean clone, since <code>experiments/</code> is gitignored.
The React app in a browser	I ran its type check and its 32 tests, and I exercised the API it talks to directly. I did not drive the UI.
A ranked session writing a leaderboard row	I played a practice session. Verifying the ranked write means putting a row in a leaderboard table, which is state I chose not to create. The code path is <code>serve.py:258-277</code> .
<code>ah exp diff</code>	I ran <code>ah exp list</code> (8 experiments) but not a diff between two of them.

Things I did not understand well enough to document

- Why the play path never calls `run_tier1` . `ah/play.py` steps cohorts

directly, so the management fee, European carry, recycling, subscription-line deferral and extension behaviour implemented in `cashflow_tier1.py` are not applied to the institution that scores. It is either a deliberate simplification whose rationale I could not find written down, or an omission.

- **Whether keeping Step 0's toy institution on live surfaces is intentional.**

`bundle.summary`, `feed.py`, `inspect.py`, `tournament.py` and `density.py` all still run it while `play.py` / `serve.py` run the real one. The 166.5 vs 76.7 divergence inside a single bundle looks like drift rather than design, but I could not find a decision either way.

- **The joinery layer (`ah/gen/joinery/`) in operational detail.** I can

describe what waypoints, the bridge and Denton reconciliation are for; I cannot tell you what a failure in them would look like or how you would diagnose it.

- ****The relationship between `ah/eval/metrics/conditional.py` 's "placeholder"**

language and what the sealed battery actually judges.** The module says a later package supersedes it; whether that package arrived is not something I established.

- **The vintage/as-of resolution rules in `ah/data/catalog.py` beyond the**

headline behaviour (immutable, pointer advances only on QC pass, `as_of` reads through pointer history). I ran `ah data status` and `ah data asof` and read their output; I did not trace the resolution logic.